# TriNetraAI — System Integration Technical Report

**Subsystem Junction Audit:** Person 1 (Data Ingestion) ↔ Person 2 (Semantic Search)

| | |
|---|---|
| **Target Subsystems:** Ingestion & Vector Search | **Date:** September 2026 |
| **Role 1 Lead:** Md Huzaifa Hayat (Data Engineering) | **Role 2 Lead:** Rahim (Semantic AI) |
| **Audit Status:** 4 Loopholes Identified & Resolved | **Classification:** Internal Engineering Review |

## 1. Executive Summary & Junction Architecture

In the TriNetraAI pipeline, **Person 1 (Data Engineering)** acts as the system's librarian—slicing raw multi-gigabyte satellite scenes into 512×512 co-registered patches and cataloging their spatial provenance. **Person 2 (Semantic AI)** acts as the translator—ingesting those patches, generating 512-dimensional multimodal embeddings using RemoteCLIP (ViT-B/32), and indexing them in Qdrant Vector DB for natural language search. The stability of the entire pipeline depends on a flawless contract between these two layers.

A rigorous line-by-line audit of ingestion/ and semantic_search/ revealed **four critical vulnerabilities/loopholes** where runtime failures, data silent-dropping, and embedding degradation occur. Below is the full technical breakdown.

## 2. Deep-Dive: Identified Loopholes & Failure Modes

### ■ Loophole 1: Filesystem Blindness (Subdirectory Traversal Mismatch)

**Root Cause:** In ingestion/tiling.py (lines 30–49), Person 1 organizes tiles into structured subdirectories partitioned by scene name: data/tiles/<scene_name>/<tile_id>.tif. However, in semantic_search/embeddings.py (lines 27–43), Person 2 uses a shallow iterator TILES_DIRECTORY.iterdir() which only inspects the immediate root folder.

| Person 1 Producer Code (ingestion/tiling.py) | Person 2 Consumer Code (semantic_search/embeddings.py) |
|---|---|
| `image_dir = root / path.stem`<br>`tile_path = image_dir / f"{tile_id}.tif"`<br>`# Writes into data/tiles/delhi_2024/tile_001.tif` | `def find_satellite_tiles():`<br>`  return sorted(p for p in TILES_DIR.iterdir()`<br>`  if p.is_file() and p.suffix == '.tif')`<br>`# FAILS: iterdir() does NOT recurse subfolders!` |

**Impact:** Person 2's tile discovery returns an empty list [], raising an unhandled FileNotFoundError: No satellite tiles were found and halting vector generation.

### ■ Loophole 2: Radiometric & Multi-Band Incompatibility (PIL vs Rasterio)

**Root Cause:** Person 1 outputs native remote sensing GeoTIFFs containing 16-bit reflectances (uint16 values up to 10,000) across 4 to 13 bands (e.g. RGB + NIR + SCL). In semantic_search/embeddings.py (line 59), Person 2 attempts to read these files using standard PIL Image.open(path).convert('RGB').

**Impact:**
1. *Multi-Band Crash:* PIL cannot parse $>4$ band GeoTIFFs (like 13-band Sentinel-2), triggering OSError: cannot identify image file.

2. *Dynamic Range Collapse:* PIL improperly converts 16-bit reflectances, outputting completely black or clipped images. Passing pitch-black arrays into RemoteCLIP corrupts the 512-D vector space, destroying search MRR accuracy.

### ■ Loophole 3: Ingestion Contract Bypass (CSV & DB Ignored)

**Root Cause:** Person 1 engineered an official export contract (export_metadata_csv() producing data/tiles_metadata.csv and maintaining SQLite tiles table with exact coordinates, CRS, resolution, and timestamps). Person 2 bypasses this registry entirely, attempting manual filesystem discovery and re-extracting bounding boxes via secondary rasterio reads.

**Impact:** Bypassing the central database breaks provenance tracking, duplicates I/O, and causes desynchronization if tiles are renamed or re-tiled.

### ■ Loophole 4: Hardcoded Remote Qdrant Host vs Air-Gapped Operation

**Root Cause:** semantic_search/vector_store.py hardcodes QDRANT_URL = 'http://localhost:6333' without a fallback.

**Impact:** In field/air-gapped deployments where external Docker containers are unavailable, vector search crashes immediately on ConnectionRefusedError rather than leveraging Qdrant's embedded in-memory/disk mode (QdrantClient(path='...')).

## 3. Architectural Resolution & Remediation Matrix

| Identified Loophole | Severity | Engineering Fix Implemented | Status |
|---|---|---|---|
| 1. Subdirectory Traversal | **HIGH** | Switch iterdir() to recursive rglob('*.tif*') and ingest directly from data/tiles_metadata.csv. | **RESOLVED** |
| 2. 16-Bit / Multi-Band PIL | **CRITICAL** | Adopt rasterio reader with 2%–98% percentile contrast normalization (shared from Role 3 preprocessor). | **RESOLVED** |
| 3. Ingestion Contract Bypass | **MEDIUM** | Bind Person 2's embedding pipeline to consume tiles_metadata.csv as single source of truth. | **RESOLVED** |
| 4. Qdrant HTTP Dependency | **MEDIUM** | Implement auto-fallback to embedded local storage QdrantClient(path='data/qdrant_local'). | **RESOLVED** |

## 4. Final Integration Verdict

By aligning the tile discovery logic, sharing the 16-bit percentile preprocessor, and enforcing the CSV metadata registry as the unified contract, the junction between Person 1 and Person 2 is now **100% airtight, resilient to nested directory layouts, and completely air-gapped ready**.